

Ερώτημα 1:

Για τον κώδικα για το DFS χρησιμοποιούμε ένα σύνορο(fringe), το οποίο είναι στοίβα. Τα στοιχεία της στοίβας είναι πλειάδες. Η κάθε μία έχει ένα state και μία λίστα με τις κινήσεις που απαιτούνται για να φτάσει σε εκείνο το state. Βγάζουμε κάθε φορά το top στοιχείο της στοίβας. Εάν δεν το έχει επισκεφθεί, τότε το προσθέτει σε λίστα με τα visited states, ελέγχει αν είναι το goal state που θέλουμε και στη συνέχεια κάνει expand το state, δημιουργώντας τα state-παιδιά του και την αντίστοιχη λίστα με τις κινήσεις μέχρι εκείνο το state, ώστε να μπουν με τη μορφή πλειάδας μέσα στη στοίβα. Ο κώδικας εκτελείται μέχρι να βρούμε goal state, όπου και επιστρέφουμε τη λίστα-μονοπάτι με τις κινήσεις μέχρι εκεί, είτε μέχρι να αδειάσει η στοίβα. Ο κώδικας εξετάζει τα states ως προς το βάθος.

Ερώτημα 2:

Παρόμοιος κώδικας με το ερώτημα 1, αλλά διαφορετικός τρόπος εξέτασης των states. Η μόνη διαφορά στον κώδικα είναι ότι το fringe είναι ουρά. Όμως η εξέταση των states γίνεται ως προς το πλάτος. Ο κώδικας εκτελείται μέχρι να βρούμε goal state, όπου και επιστρέφουμε τη λίστα-μονοπάτι με τις κινήσεις μέχρι εκεί, είτε μέχρι να αδειάσει η ουρά.

Ερώτημα 3:

Παρόμοιος κώδικας με τα παραπάνω ερωτήματα, όμως πλέον το fringe είναι ουρά προτεραιότητας. Δηλαδή, δέχεται την πλειάδα που περιγράψαμε παραπάνω και μια μεταβλητή με το άθροισμα του κόστους κάθε κίνησης από το αρχικό state μέχρι το τωρινό state, η οποία αποτελεί τη προτεραιότητα της. Τα στοιχεία ταξινομούνται στην ουρά προτεραιότητας, σε κάθε εισαγωγή στοιχείου, από το μικρότερο στο μεγαλύτερο. Δηλαδή, πρώτο θα είναι το στοιχείο με τη μικρότερη τιμή στην προτεραιότητα και τελευταίο αυτό με τη μεγαλύτερη. Έτσι, κάνουμε expand κάθε φορά το state με το μικρότερο άθροισμα του κόστους των κινήσεων που απαιτούνται για να φτάσουμε σε αυτό. Άρα βρίσκουμε το μονοπάτι από το αρχικό state σε ένα goal state με το μικρότερο κόστος κινήσεων. Ο κώδικας εκτελείται μέχρι να βρούμε goal state, όπου και επιστρέφουμε τη λίστα-μονοπάτι με τις κινήσεις μέχρι εκεί, είτε μέχρι να αδειάσει η ουρά προτεραιότητας.

Ερώτημα 4:

Παρόμοιος κώδικας με το ερώτημα 3, η μόνη διαφορά είναι ότι η προτεραιότητα του κάθε state είναι ίση με το άθροισμα του κόστους των κινήσεων από το αρχικό state μέχρι το τωρινό state και της τιμής μίας ευρετικής συνάρτησης που υπολογίζει μία εκτίμηση του κόστους των κινήσεων από το τωρινό state μέχρι το κοντινότερο goal state. Ο κώδικας εκτελείται μέχρι να βρούμε goal state, όπου και επιστρέφουμε τη λίστα-μονοπάτι με τις κινήσεις μέχρι εκεί, είτε μέχρι να αδειάσει η ουρά προτεραιότητας.

Ερώτημα 5:

Το state πλέον είναι μία πλειάδα με τις συντεταγμένες του και μία λίστα με 4 στοιχεία(ένα για κάθε γωνία) με τιμή True(αν έχουμε επισκεφθεί τη γωνία στην διαδρομή μας μέχρι εκεί) ή False(αν δεν την έχουμε επισκεφθεί). Ένα state είναι goal state αν όλες οι γωνίες έχουν γίνει visited στη διαδρομή μέχρι εκεί,

δηλαδή αν δεν υπάρχει καμία τιμή False στην προαναφερθείσα λίστα. Η συνάρτηση getSuccessors επιστρέφει μία λίστα με όλους τους state-διαδόχους του τωρινού state. Ο διάδοχος είναι μία πλειάδα με το state, τη κίνηση που απαιτείται για να πάει στο state και το κόστος της κίνησης. Το state είναι μία πλειάδα με τις συντεταγμένες του και τη προαναφερθείσα λίστα. Επειδή όμως ένας successor μπορεί να είναι γωνία, ελέγχουμε αν είναι και βάζουμε σε νέα δική του λίστα με τις γωνίες τη τιμή True στην θέση που αντιστοιχεί η γωνία.

Ερώτημα 6:

Η ευρετική συνάρτηση cornersHeuristic υπολογίζει την απόσταση από το state που βρισκόμαστε μέχρι ένα goal state(δηλαδή μια γωνία). Η συνάρτηση υπολογίζει την απόσταση του state από κάθε γωνία, με τη συνάρτηση manhattanDistance, που δεν βρίσκεται στη λίστα με τις visited γωνίες κι επιστρέφει τη μεγαλύτερη από αυτές. Η ευρετική είναι παραδεκτή(admissible) διότι επιστρέφει ένα κάτω φράγμα στην μικρότερη δυνατή απόσταση του state από ένα goal state, αφού επιστρέφουμε τη μεγαλύτερη απόσταση του state από ένα goal state. Η ευρετική είναι και συνεπής(consistent) διότι εφόσον οι διαθέσιμες κινήσεις είναι πάνω, κάτω, δεξιά, αριστερά, η τιμή της manhattanDistance ενός state με αυτή του επόμενου state θα διαφέρει κατά 1. Εφόσον όλες οι κινήσεις έχουν κόστος ίσο με 1, τότε θα ισχύει $h(a) \leq c(a,b) + h(b)$, όπου a το state που βρισκόμαστε και b το επόμενο του.

Ερώτημα 7:

Η ευρετική συνάρτηση foodHeuristic υπολογίζει την απόσταση από το state που βρισκόμαστε μέχρι ένα goal state(δηλαδή ένα food). Η συνάρτηση υπολογίζει την απόσταση του state από κάθε food με τη συνάρτηση mazeDistance κι επιστρέφει τη μεγαλύτερη από αυτές. Η ευρετική είναι παραδεκτή(admissible) διότι επιστρέφει ένα κάτω φράγμα στην μικρότερη δυνατή απόσταση του state από ένα goal state, αφού επιστρέφουμε τη μεγαλύτερη απόσταση του state από ένα goal state. Η ευρετική είναι και συνεπής(consistent) διότι η τιμή της mazeDistance ενός state με αυτή του επόμενου state θα διαφέρει κατά 1. Εφόσον όλες οι κινήσεις έχουν κόστος ίσο με 1, τότε θα ισχύει $h(a) \leq c(a,b) + h(b)$, όπου a το state που βρισκόμαστε και b το επόμενο του.

Ερώτημα 8:

Το επιθυμητό μονοπάτι για το κοντινότερο food στο state το βρίσκουμε με την κλήση του αλγορίθμου αναζήτησης που υλοποιήσαμε στο ερώτημα 2, τον BFS.